Distributed Job Scheduler

Made By Harish V [RA2311003050007]

Project Links

- **Live Preview (Vercel):** https://jobshedulerproject.vercel.app/
- **GitHub Repository:** https://github.com/ZeonArc/jobshedulerproject
- **Google Drive (Source Code Zip):** https://drive.google.com/file/d/13M1vYt3NufMsNto-ggo5ckSffBQTLiN8/view?usp=sharing

---

1. Setup Instructions

To run this project locally, ensure you have Node.js (v18+) and PostgreSQL installed.

1. Clone and Database Setup

- Ensure PostgreSQL is running on `localhost:5432`.
- Create a database named `jobscheduler`.

2. Backend Setup

```
cd backend
npm install
# Create a .env file with
DATABASE_URL=postgresql://user:password@localhost:5432/jobscheduler
npx prisma db push
npx prisma generate
npm run dev
```

(The backend runs on port 4000)

3. Frontend Setup

```
cd frontend
npm install
npm run dev
```

(The frontend runs on port 3000)

4. Testing

```
cd backend
npx jest
```

2. Architecture Diagram

We utilized a decoupled Service-Worker Architecture with real-time WebSockets.

```
graph TD
    Client[Next.js Frontend Dashboard] -->|HTTP REST + JWT| API(Node.js / Express
API)
    Client <-->|Socket.io Live Events| API

    API -->|Prisma ORM| DB[(PostgreSQL)]

    Worker1[Distributed Worker 1] <-->|SKIP LOCKED Polling| DB
    Worker2[Distributed Worker 2] <-->|SKIP LOCKED Polling| DB

    Worker1 -->|Execute Webhooks| ExternalAPI(External Services)
    Worker2 -->|Execute Webhooks| ExternalAPI

    Worker1 -->|Emit Progress| API
```

3. Entity Relationship (ER) Diagram

The database strictly utilizes relational constraints to enforce data integrity.

```
erDiagram
    User ||--o{ Project : owns
    Project ||--o{ Queue : contains
    Queue ||--o{ Job : schedules
    Worker ||--o{ Job : processes
    Job ||--o{ Job : depends_on

    User {
        String id PK
        String email
        String passwordHash
    }
    Project {
        String id PK
        String name
        String userId FK
    }
    Queue {
        String id PK
        String name
        Int concurrencyLimit
        Int rateLimit
        Boolean isPaused
        String projectId FK
```

```
    }
    Job {
        String id PK
        String status "queued, running, completed, failed, etc"
        Json payload "Webhook URL, Body, Method"
        Int priority
        Int progress
        String queueId FK
        String workerId FK
    }
    Worker {
        String id PK
        String status
        DateTime lastHeartbeatAt
    }
```

---

4. Design Decisions & Major Trade-Offs

1. Database-Backed Queue vs. Redis

- **Decision:** We used PostgreSQL with `FOR UPDATE SKIP LOCKED` for the job queue instead of a memory store like Redis/BullMQ.
- **Trade-Off:** PostgreSQL introduces slightly higher latency (milliseconds) compared to Redis memory access. However, it completely eliminates the need for secondary infrastructure (no Redis server to host). `SKIP LOCKED` natively solves distributed concurrency, preventing two workers from claiming the same job without requiring complex distributed locks.

2. DAGs via Relational Adjacency List

- **Decision:** Workflow Dependencies (DAGs) are managed natively via a many-to-many self-relation on the `Job` model (`dependencies` and `dependents`).
- **Trade-Off:** Querying deep nested graphs in standard SQL can be expensive. However, by strictly utilizing `waiting` and `completed` status hooks in the Worker to trigger unlock cascades, we flatten the query requirement at runtime.

3. Long-Polling vs Event-Driven Workers

- **Decision:** The distributed workers use an adaptive long-polling loop with exponential backoff.
- **Trade-Off:** True event-driven workers (e.g. Postgres LISTEN/NOTIFY) offer instant reaction times. However, LISTEN/NOTIFY requires persistent connections which scales poorly. Adaptive long-polling provides sub-second latency while keeping database connections ephemeral and highly scalable.

4. Mock AI vs. OpenAI API

- **Decision:** The "AI Failure Analysis" feature utilizes a local deterministic analyzer rather than calling OpenAI.
- **Trade-Off:** While real LLM analysis is deeply insightful, external APIs introduce cost, rate limits, and network failure points during academic/evaluator testing.

---

5. API Documentation

All endpoints are secured via JWT `Bearer` token in the `Authorization` header.

Authentication

- `POST /api/auth/register` - Register a new user
- `POST /api/auth/login` - Authenticate and receive a JWT

Projects

- `GET /api/projects` - List all user projects
- `POST /api/projects` - Create a project

Queues

- `POST /api/queues` - Provision a queue (`concurrencyLimit`, `rateLimit`)
- `POST /api/queues/:id/pause` - Halt a queue
- `POST /api/queues/:id/resume` - Resume a queue
- `GET /api/queues/:id/stats` - Fetch aggregate queue statistics

Jobs

- `GET /api/jobs?projectId=x` - Fetch jobs (with dependencies)
- `POST /api/jobs` - Enqueue a job (`url`, `priority`, `dependsOn`, `retryStrategy`)
- `POST /api/jobs/:id/retry` - Manually requeue a Dead Letter job
